STATEMENT OF WORK (SOW): Enterprise Financial Analytics Dashboard

Project Name: FinDash Analytics UI - Core Production Architecture
Total Estimated Cost: $1,600
Estimated Timeline: 8-12 Weeks

1.Summary

We are developing a B2B SaaS financial dashboard tailored for CFOs and enterprise finance teams. The application demands a desktop-class, highly responsive user experience, featuring data-dense views, deep filtering, and interactive visualizations. **Objective:** The Contractor will architect and implement a production-ready, highly interactive, accessible, and performant frontend utilizing the Next.js App Router. This engagement focuses on delivering scalable persistent nested layouts, bulletproof URL state synchronization, buttery-smooth rendering of massive datasets, and comprehensive test coverage.

---

2. Scope of Work

The Contractor will be responsible for delivering the complete frontend architecture and the following core pages:

Phase 1: Foundational Architecture & Persistent Layouts

Architect the app/ directory to support a persistent global sidebar, contextual top navigation, and localized sub-navigation.

- Guarantee seamless navigation between primary routes (e.g., /dashboard/overview to /dashboard/transactions) without unmounting global navigation or losing local component state.
- Implement active-state styling for deeply nested route navigation using native Next.js router hooks.

Phase 2: Complex UI Implementation

- **The Overview (/dashboard/overview):** Develop a modular, widget-based layout allowing users to drag, drop, and resize chart widgets (utilizing dnd-kit). Persist layout state globally.

- **The Transaction Ledger (/dashboard/transactions):** Build a high-performance data grid capable of rendering 10,000+ rows. Must implement viewport virtualization (TanStack Virtual) to guarantee scrolling is locked at a strict 60fps.
- **Report Builder (/dashboard/reports/create):** Construct a multi-step configuration wizard utilizing client-side state management with animated step transitions.

Phase 3: Advanced State Management & URL Synchronization

- Implement complex filtering for the Transaction Ledger (Date range, Amount min/max, Category multi-select, Search string).
- Engineer a bi-directional synchronization system between the client-side filter state and the URL Search Parameters (e.g., ?start=2026-03-01&end=2026-03-19&category=software).
- Ensure URLs are fully shareable, perfectly reconstructing the UI state for external users without triggering full-page reloads or losing input focus during query updates.

Phase 4: Polish, Animation, and Accessibility

- Integrate layout animations for page transitions and staggering entrance effects for list items.
- Ensure dynamic UI elements (dropdowns, modals) utilize layout animations to resize smoothly, preventing jarring layout shifts.
- Implement enterprise-grade accessibility (WCAG 2.1 AA compliance), ensuring full keyboard navigability and screen-reader support via accessible UI primitives.
- Develop a robust Light/Dark mode toggle using CSS variables with a strict "Zero FOUC" (Flash of Unstyled Content) requirement on initial server load.

Phase 5: Testing, QA & Handover

- Write Unit tests for complex utility functions (e.g., URL state parsers) using Jest/Vitest.
- Write End-to-End (E2E) tests for critical user journeys (e.g., Report Builder flow, filtering the transaction ledger) using Playwright.
- Perform a comprehensive Lighthouse performance audit.

---

3. Technical Specifications

The deliverables must strictly adhere to the following technology stack:

- **Framework:** Next.js (App Router, latest stable)
- **Styling:** Tailwind CSS + shadcn/ui (or Radix UI primitives)
- **State / URL Management:** nuqs (Type-safe search parameters) and Zustand (Global UI state)
- **Data Visualization & Tables:** Recharts (or Visx), TanStack Table, and TanStack Virtual
- **Animation:** Framer Motion
- **Testing:** Vitest (Unit) and Playwright (E2E)

---

4. Deliverables & Acceptance Criteria

Upon completion, the Contractor must provide a production-ready codebase containing the following deliverables for review:

1. **Layout Hierarchy:** A documented app/(dashboard) route group demonstrating optimized rendering and persistent UI management.
2. **Filter Component Logic:** Custom hooks demonstrating debounced user input mapped seamlessly to URL state without unnecessary React re-renders.
3. **Data Grid Implementation:** The virtualized transaction table component proving 60fps scrolling, sorting, and row selection performance on large datasets.
4. **Test Suite & Audits:** * Passing E2E test suite covering core workflows.
    - A Lighthouse performance score of 90+ on desktop for all core dashboard routes.
    - A WCAG accessibility compliance report utilizing tools like Axe.

---

5. Project Timeline & Payment Schedule

Cost Allocation:
- Frontend Development (Scope defined in this SOW): $350 USD
- Remaining Budget (Backend, Design, PM, etc.): $1,250 USD

Payment Terms for Frontend Contractor:
- Total Compensation: $350 USD
- Payment Schedule:
• 30% ($105) upfront
• 40% ($140) after Phase 2 completion
• 30% ($105) upon final delivery and acceptance